

DataForge: Visualizing Fixed Hebbian Memory vs. Transformer KV-Cache

An interactive educational artifact inspired by recent research on structured memory for long-context language models

Introduction

Modern Large Language Models (LLMs) rely on the Transformer architecture, which stores previously processed tokens inside a **Key-Value (KV) cache** during inference. While this mechanism enables attention over long contexts, it introduces an important limitation: the required memory grows linearly with the number of processed tokens. As context lengths increase from thousands to hundreds of thousands of tokens, KV-cache memory becomes a major bottleneck for deployment.

Recent research has explored alternative memory mechanisms that replace growing token storage with **fixed-size learned memories**. One such direction is presented in *The Dragon Hatchling: Learning to Reason with Structured Memory* (Pathway, 2025), which proposes a biologically inspired memory architecture based on Hebbian updates.

This project, **DataForge**, is an educational visualization that helps learners understand the difference between these two memory paradigms. Rather than reproducing the Dragon Hatchling architecture, the artifact presents a simplified toy model illustrating one central concept: **a growing KV-cache versus a fixed-size associative memory**.

What the Artifact Demonstrates

The artifact combines Python notebooks with a lightweight interactive web interface.

The computational pipeline first generates synthetic sparse activation patterns using a reproducible random seed. A simple Hebbian learning rule updates a fixed-size synaptic matrix while analytical equations estimate the memory required by a Transformer KV-cache for the same context length.

The resulting data are exported as JSON files and visualized through an interactive webpage.

The interface allows users to:

- Adjust context length from 0 to approximately 128k tokens.
- Compare Transformer KV-cache memory against fixed Hebbian memory.
- Observe how KV-cache memory increases linearly while Hebbian memory remains constant.
- Explore a simplified visualization of Hebbian matrix updates.
- View precomputed metrics including Frobenius norm, sparsity, and memory footprint.

Unlike static figures, every visualization updates interactively, making the memory trade-off immediately observable.

Architecture

The repository is organized into three major components.

Python source (src/)

Implements the simplified Hebbian learning model, simulation logic, evaluation utilities, and helper functions.

Jupyter notebooks (notebooks/)

Generate synthetic experiments and export all visualization data into JSON files.

Interactive artifact (artifact/)

A browser-based interface built with HTML, CSS, and JavaScript. Rather than recomputing the simulation inside the browser, it loads the precomputed JSON outputs and provides interactive exploration.

This separation keeps the frontend lightweight while preserving reproducibility of the computational pipeline.

Educational Scope and Limitations

This project is intentionally educational rather than experimental.

Several important simplifications should be noted:

- It is **not an implementation of the Dragon Hatchling architecture**.
- Token activations are synthetically generated rather than extracted from a language model.
- Transformer memory values are computed analytically rather than measured from a running model.
- The Hebbian update rule is a simplified associative learning demonstration intended for visualization.

Accordingly, the artifact should be viewed as a teaching aid for understanding architectural trade-offs rather than as a benchmark or reproduction of current research.

References

Primary inspiration

Pathway. *The Dragon Hatchling: Learning to Reason with Structured Memory*. arXiv:2509.26507, 2025.

<https://arxiv.org/abs/2509.26507>

Additional background

Dao, T., Fu, D., Ermon, S., Rudra, A., & Ré. C. *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*. NeurIPS, 2022.

<https://arxiv.org/abs/2205.14135>

Ramsauer, H., Schöfl, B., Lehner, J., et al. *Hopfield Networks is All You Need*. ICLR, 2021.

<https://arxiv.org/abs/2008.02217>

Repository: <https://github.com/pnkj006/bdh-vs-kv-cache-artifact>

This artifact was developed for the **Pathway × Prime DataForge Hackathon** as an educational visualization of memory architectures in modern language models.